

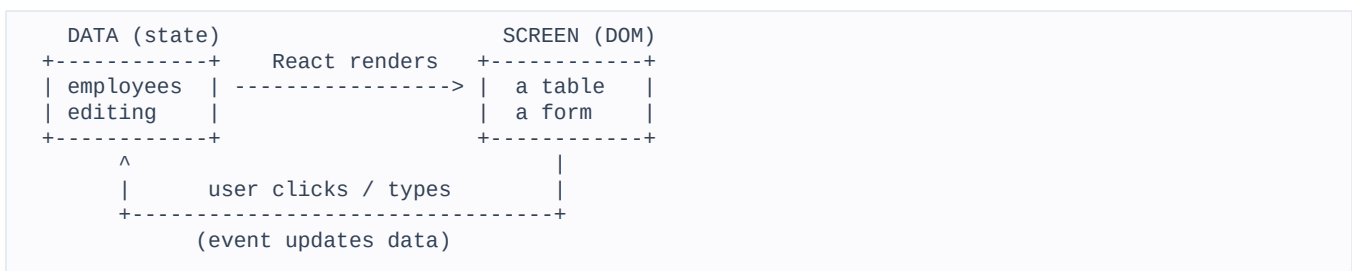
Understanding the React Frontend

A beginner's guide to the Employee POC single-page app

This guide explains the React concepts used in the **Employee POC** frontend, written for a beginner. Each concept is tied to the actual files in your project: `src/main.jsx`, `src/App.jsx`, `src/components/EmployeeForm.jsx`, `src/components/EmployeeTable.jsx`, and `src/api/employeeApi.js`. Open each file alongside this guide as you read.

1. The mental model: what React does

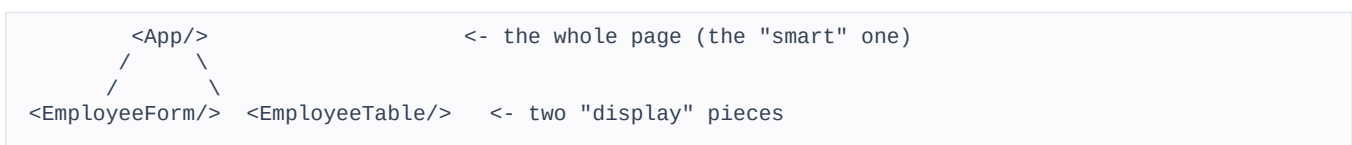
React's one big idea: **your UI is a function of your data**. You don't manually update the page ("find this cell, change its text"). Instead you describe what the screen should look like for the current data, and when the data changes React re-runs your functions and updates the screen for you.



Change the data → React redraws. That loop is the whole game.

2. Components: the building blocks

A **component** is just a JavaScript function that returns markup (JSX). Your app has three, arranged in a tree:



A component returns **JSX** — HTML-like syntax inside JavaScript:

```
function EmployeeTable() {
  return <div className="card"><h2>Employees</h2></div>
}
```

It is `className`, not `class` (because `class` is a reserved word in JavaScript). JSX is not HTML — it is JavaScript that *looks* like HTML and compiles down to function calls.

3. The entry point: how the app boots

`main.jsx` is where React attaches to the page:

```
ReactDOM.createRoot(document.getElementById('root')).render(<App />)
```

index.html		main.jsx		Your components
+-----+		+-----+		+-----+
<div id=root>	<-----	createRoot	<--renders--	<App/>
</div>		.render()		+-----+
+-----+		+-----+		

`index.html` has one empty `<div id="root">`. React takes control of that div and renders `<App/>` (and everything inside it) into it. That is why it is called a **Single Page App** — one HTML file, and React swaps the contents.

`<React.StrictMode>` is a dev-only helper that double-checks your code. It intentionally runs some code twice in development — harmless, and gone in production.

4. State: data that changes over time (useState)

When data can change and the screen must react, it goes in **state**. You create state with the `useState` hook. From the top of `App.jsx`:

```
const [employees, setEmployees] = useState([]) // the list, starts empty
const [editing, setEditing] = useState(null) // which row is being edited, or null
const [error, setError] = useState('')
const [loading, setLoading] = useState(false)
```

Reading that line:

```
const [ employees , setEmployees ] = useState( [ ] )
      |           |               |
      current value function to   the INITIAL value
      (read it)     change it     (only used on first render)
```

The rule that trips up every beginner: never assign to the variable directly. Always call the setter.

```
employees = [...] // WRONG - React won't notice, screen won't update
setEmployees([...]) // RIGHT - tells React "data changed, please redraw"
```

Calling `setEmployees(...)` does two things: updates the value **and** schedules a re-render (React re-runs the `App` function to redraw the screen).

5. Props: passing data down to children

Components talk to their children through **props** (properties) — like function arguments. In `App`'s JSX:

```
<EmployeeTable
  employees={employees} // data going DOWN
  loading={loading}
  onEdit={handleEdit} // a function going DOWN (so the child can call back UP)
  onDelete={handleDelete}
/>
```

And `EmployeeTable` receives them as one object:

```
function EmployeeTable({ employees, loading, onEdit, onDelete }) { ... }
```

```

+----- App (owns the data) -----+
|                                     |
props | employees, onEdit, onDelete   props | editing, onSave, onCancel
      |                               |
      v                               v
    <EmployeeTable>                 <EmployeeForm>

```

Golden rule of React data flow — it is one-directional:

```

Data flows DOWN via props      vvv
Events flow UP via callbacks   ^^^

```

A child cannot reach up and change the parent's state directly. Instead the parent hands down a function (`onEdit`), and the child *calls* it. This is called "**lifting state up**" — the shared data (`employees`) lives in the parent, and children ask the parent to change it.

6. A concrete walkthrough: "Delete an employee"

Let's trace one full loop through your code so the pieces connect:

```
1. User clicks "Delete" in a table row
   EmployeeTable.jsx: onClick={() => onDelete(emp.id)}
                        |
                        | calls the function App passed down
                        v
2. App.jsx: handleDelete(id) runs
   |- window.confirm("Delete this employee?")
   |- await deleteEmployee(id) -----+ (calls the API layer)
   |_ await loadEmployees()           |
                                       v
3. employeeApi.js: axios sends DELETE /api/employees/5 to Spring Boot
                                       |
                                       |
4. loadEmployees() runs getEmployees() -> setEmployees(freshList)
                                       |
                                       | setEmployees triggers a RE-RENDER
                                       v
5. App re-runs -> passes the new (shorter) list to EmployeeTable
   -> the deleted row is gone from the screen
```

Notice you never touched the DOM. You changed **state** (`setEmployees`), and React redrew the table. That is the pattern for *everything*.

7. Side effects: talking to the outside world (useEffect)

Rendering should be "pure" — just compute the UI. But you need to *fetch data from the server*, which is a **side effect**. That is what `useEffect` is for. In `App.jsx`:

```
useEffect(() => {
  loadEmployees() // run this...
}, [loadEmployees]) // ...after render, when this list changes
```

```
App renders for the first time
|
v
screen shows (empty table)
|
v
useEffect fires --> loadEmployees() --> server --> setEmployees(data)
|
v
App re-renders --> table now shows the employees
```

The second argument `[...]` is the **dependency array** — it controls *when* the effect runs:

- `[]` → run **once**, after the first render (most common for "load on page open").
- `[loadEmployees]` → run again if `loadEmployees` ever changes.
- (omitted entirely) → run after **every** render (rarely wanted — can cause infinite loops).

Why does `useCallback` wrap `loadEmployees`? Every time `App` re-renders, all functions inside it are recreated (a new object in memory). If `loadEmployees` were brand-new each render, `useEffect` would think its dependency changed and refetch endlessly. `useCallback(fn, [])` keeps the *same* function object across renders, breaking that loop. Don't stress about it early — just know why it is there.

8. Controlled forms: React owns the input values

Open `EmployeeForm.jsx`. This is the trickiest beginner concept, so go slow.

In plain HTML, the input box remembers its own text. In React, we make **state** the single source of truth and bind the input to it. This is a **controlled input**:

```
const [form, setForm] = useState(EMPTY) // one object holds all 6 fields

<input
  name="firstName"
  value={form.firstName}           // (1) input DISPLAYS what is in state
  onChange={handleChange}         // (2) every keystroke UPDATES state
/>
```

```

You type "A"
  |
  v
onChange fires --> handleChange --> setForm({...form, firstName: "A"})
  |                               |
  |                               v
  |                               state updates
  |                               |
  |                               v
input re-renders showing value={form.firstName} ("A")
```

It is a tiny loop on every keystroke: **type** → **update state** → **re-render** → **input shows new state**. The box always mirrors state, so `form` always has the latest values — ready to submit.

`handleChange` is written once for all six inputs using the input's `name`:

```
const handleChange = (e) => {
  const { name, value } = e.target // e.g. name="email", value="a@b.com"
  setForm((prev) => ({ ...prev, [name]: value })) // update just that one field
}
```

The `{ ...prev, [name]: value }` means "copy all existing fields, then overwrite the one that changed." The `...` is the **spread operator** — copying instead of mutating is a core React habit (never change state objects in place).

Submitting:

```
const handleSubmit = (e) => {
  e.preventDefault() // stop the browser's default page reload on form submit
  onSave({ ...form, salary: Number(form.salary) }) // hand data UP to App
  if (!editing) setForm(EMPTY) // clear the form after adding
}
```

`e.preventDefault()` is essential — without it, the HTML form reload wipes your whole app.

9. One form, two jobs: Add vs. Edit

Your form is reused for both adding and editing. It decides its mode from the `editing` prop:

```
editing === null -> "Add Employee" button, empty fields, calls POST
editing === {...} -> "Edit #5" button, pre-filled, calls PUT
```

```
<h2>{editing ? `Edit Employee #${editing.id}` : 'Add Employee'}</h2>
<button>{editing ? 'Update' : 'Add Employee'}</button>
```

`{ condition ? A : B }` inside JSX is **conditional rendering** — a plain JavaScript ternary embedded in markup with `{ }`. The braces mean "drop out of JSX and run some JavaScript here."

The clever bit in App.jsx:

```
<EmployeeForm key={editing ? editing.id : 'new'} editing={editing} ... />
```

That `key` prop is a trick. When `key` changes (you click Edit on row 5 → key becomes 5), React **throws away the old form and builds a fresh one**, so its internal `useState(EMPTY)` re-initializes with the employee's data. Without this, the form's local state would not refresh when you switch records.

10. Rendering lists: .map() and key

In `EmployeeTable.jsx`, you turn an array of employees into an array of table rows:

```
{employees.map((emp) => (  
  <tr key={emp.id}>  
    <td>{emp.firstName}</td>  
    <td>{emp.email}</td>  
    ...  
  </tr>  
))}
```

employees (array)		rendered rows
+-----+	.map()	+-----+
{id:1, ...}	----->	<tr>...</tr>
{id:2, ...}		<tr>...</tr>
{id:3, ...}		<tr>...</tr>
+-----+		+-----+

`.map()` is just JavaScript's array method — it transforms each item into a `<tr>`. The `key={emp.id}` is required by React: it is a stable ID so React can tell *which* row is which when the list changes (add / delete / reorder), and update efficiently. Always use a unique, stable value (a database `id`) — never the array index if the list can change.

11. Conditional rendering: loading / empty / data

Still in the table, you show different things depending on state:

```
{loading ? (  
  <tr><td colSpan="8">Loading...</td></tr>  
) : employees.length === 0 ? (  
  <tr><td colSpan="8">No employees yet. Add one above.</td></tr>  
) : (  
  employees.map(...) // the real rows  
)}
```

```
loading?  --yes-->  show "Loading..."  
|  
no  
v  
list empty?  --yes-->  show "No employees yet"  
|  
no  
v  
show the rows
```

This is how React apps handle the three states every data screen has: *fetching*, *empty*, and *has data*. Handling all three is what makes a UI feel finished.

12. The API layer: keeping network code separate

`employeeApi.js` is not a component — it is a plain module that wraps `axios` (an HTTP library). One function per backend endpoint:

```
const api = axios.create({ baseURL: 'http://localhost:8080/api/employees' })
```

```

export const getEmployees = ()      => api.get('')
export const createEmployee = (employee) => api.post('', employee)
export const updateEmployee = (id, emp) => api.put(`/${id}`, emp)
export const deleteEmployee = (id)     => api.delete(`/${id}`)

```

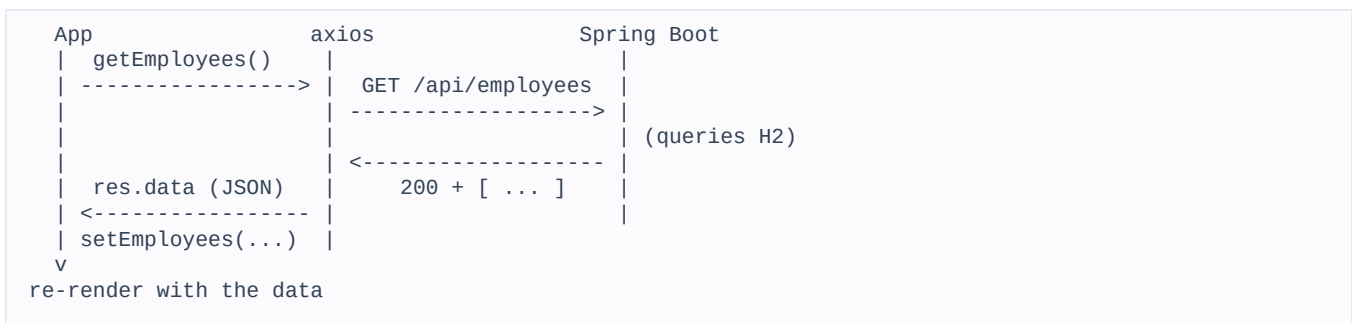
Why a separate file? **Separation of concerns** — components worry about *display*; this file worries about *the network*. If your API URL changes, you edit one file, not every component.

These functions return **Promises** (a value that arrives *later*, once the server responds). That is why **App** uses **async/await**:

```

const res = await getEmployees() // "wait here until the server replies"
setEmployees(res.data)           // res.data is the JSON array from Spring Boot

```



try/catch around these calls is how you handle failures (server down, validation error) — that is what fills your red error banner via **setError(...)**.

13. Putting it all together



The five concepts to truly internalize (everything else builds on these):

1. **State** = data that, when changed via its setter, redraws the screen.
2. **Props** = data / functions passed *down* from parent to child.
3. **Events flow up** via callback functions; **data flows down** via props.
4. **useEffect** = do something outside React (like fetch data), controlled by its dependency array.
5. **Controlled inputs** = form values live in state, bound with `value` + `onChange`.

14. Quick glossary

Term	Meaning
Component	A function that returns JSX (a piece of UI).
JSX	HTML-like syntax inside JavaScript.
Props	Inputs passed from a parent component to a child.
State	Data a component remembers; changing it re-renders the UI.
Hook	A special function (useState, useEffect, useCallback) that adds features to a component.
Re-render	React re-running a component function to update the screen.
Controlled input	A form field whose value is driven by state.
Side effect	Work outside pure rendering, e.g. fetching data (useEffect).

Promise / async / await	How JavaScript handles values that arrive later (network calls).
Lifting state up	Keeping shared data in the parent and passing it + updater functions down.